

BaiZe: Building an Efficient 2B Hybrid State-Space Agentic Language Model from Scratch via Budget-Constrained Architecture and Hyper-Parameter Search

First Author¹, Second Author¹

Company / Institution

email@domain.com

City, Country

Third Author²

Company / Institution

email@domain.com

City, Country

Abstract—Large language models (LLMs) exhibit strong reasoning and tool-use skills, yet their computational footprint makes most agentic models impractical for edge, embedded, and cost-sensitive electronic design automation (EDA) deployments. We present BaiZe, a ≈ 2.2 -billion-parameter agentic language model built *from scratch* on a fixed compute budget around a *hybrid* state-space backbone. BaiZe adopts a Nemotron-H-style sequential hybrid architecture—56 layers interleaving Mamba-2 state-space layers, four grouped-query-attention layers, and MLP layers—whose linear-time decoding is well suited to inference-heavy, cost-sensitive agents, and is trained with the NVIDIA NeMo recipe on Megatron-Core. The architecture is chosen through a matched from-scratch comparison against a dense transformer counterpart, in which the hybrid attains lower loss and faster inference with fewer parameters at a modest training-throughput cost. We then run a staged, budget-constrained hyper-parameter search (S1–S4) on $8\times H100$ over the final hybrid backbone; all loss, throughput, and inference measurements are re-evaluated on this backbone and reported as [TBD] pending full-scale training. We release the complete recipe and training command to support the reproducible construction of efficient agentic small language models.

Index Terms—small language model, agentic LLM, hybrid architecture, state-space model, budget-constrained search, efficient inference

I. INTRODUCTION

Large language models have recently demonstrated agentic capabilities—planning, tool invocation, and long-horizon reasoning—that make them attractive cores for intelligent assistants spanning chip design, verification, and other domains of electronic design automation. However, state-of-the-art agent backbones are routinely 7B parameters or larger, demanding high-end accelerators and memory footprints that are incompatible with edge, embedded, and cost-sensitive deployments where inference latency and energy are first-class constraints.

Recent small language models (SLMs, < 2 B) have closed much of the gap with larger counterparts on single-turn benchmarks, but building such a model *from scratch* remains an under-documented engineering problem. Every choice—architecture, optimizer, learning-rate schedule, data mixture—

interacts with every other, and a naive grid search is prohibitively expensive. Our goal is therefore twofold: (i) to construct, from scratch and on a strictly limited compute budget, an end-to-end multimodal agentic system—an efficient hybrid language core paired with a custom vision encoder, aligned into a semiconductor-domain agent—and (ii) to expose the concrete, reproducible decisions that lead to it.

a) BaiZe.: We present BaiZe, an end-to-end multimodal agentic system assembled from scratch across five stages: (i) pre-training a ≈ 2.2 B-parameter hybrid (state-space + attention) LLM, (ii) post-training it for agentic capability, (iii) pre-training a custom vision encoder, (iv) aligning the two into a multimodal model, and (v) adapting the result to the semiconductor domain. Three design pillars organise our work:

- **Architecture.** A Nemotron-H-style sequential hybrid language backbone—56 layers interleaving Mamba-2 state-space layers, four grouped-query-attention layers, and MLP layers—giving linear-time decoding for inference-heavy agents, paired with a from-scratch vision encoder.
- **Reduced-horizon proxy search.** We exploit short, cheap proxy runs—in the spirit of maximal-update parameterization (μ P) transfer [1]—so that the relative ranking of candidate hyper-parameters discovered at small scale carries over to the full model without re-tuning at scale.
- **Budget-constrained stage-wise pipeline.** Each of the five stages is trained under a hard GPU-hour budget, with a staged search protocol (S1–S4) for the language backbone (numeric results reported as [TBD] pending re-measurement).

We hope BaiZe serves as a minimal, reproducible blueprint for building efficient agentic SLMs from scratch, and that our search methodology is of practical value to cost-bounded training efforts in the EDA and embedded-AI communities.

II. BACKGROUND AND RELATED WORK

A. Small Language Models and Agentic Reasoning

Parameter-efficient SLMs (< 2 B) have recently narrowed the gap with larger models on mathematical and commonsense reasoning. MiniCPM [2] and its successors popularised the WSD learning-rate schedule and domain-weighted data mixing; Qwen [3] and Gemma [4] demonstrate that strong code- and math-enriched corpora can lift small models substantially. Most of these efforts, however, treat the final trained model as the deliverable and report comparatively little about the *search* that produced it. Complementarily, agentic instruction-tuning datasets (e.g., tool-use and multi-step reasoning corpora) [5] have been shown to elicit planning and tool-invocation behaviour, yet are rarely paired with a from-scratch pre-training recipe for the 1B regime.

B. Proxy Search and Hyper-Parameter Transfer

Maximal-update parameterization (μ P) [1], [6] preserves training dynamics across model widths, enabling optimal hyper-parameters to be discovered on small “proxy” models and transferred to the full-sized network; reduced-horizon ranking strategies in the same spirit [2] trade a small number of expensive full runs for many cheap short runs that reliably order candidate hyper-parameters. Both are especially valuable for budget-bounded builders, because they convert expensive full-scale sweeps into cheap small-scale ones; we adopt this short-horizon proxy ranking on the final hybrid backbone, transferring hyper-parameters tuned in short runs to the full configuration.

C. Hybrid and State-Space Architectures

State-space models (SSMs) such as Mamba [7] and Mamba-2 [8] replace full attention with a recurrent scan update, offering linear-time decoding that suits inference-bound deployments. Hybrid designs interleave a small number of attention layers among SSM/MLP layers—notably Nemotron-H [9] and Hymba [10]—to recover long-range context while retaining fast decode. Released Megatron-Core variants of these families are scarce below 2B, leaving their small-scale trade-offs under-documented; our controlled 2B comparison (Section IV-A) addresses precisely this gap.

D. Efficient Training and WSD Schedules

Warmup–Stable–Decay (WSD) schedules [2] decouple long, constant-rate stable training from a short annealing phase, offering a predictable knob for late-stage adaptation. Recent work combines WSD with domain-weighted decay-time data mixing and, in some systems, optimizer switching during decay [11]. Megatron-Core and its downstream wrappers (e.g., Megatron-SWIFT) provide fused kernels and FP8/selective-recompute primitives [12] that make 1B-scale from-scratch training practical on a small GPU pool—the setting we target.

E. LLMs for Electronic Design Automation

A growing line of work applies LLM agents to chip design tasks such as register-transfer-level generation, verification, and bug triage [13], [14]. These applications motivate a cheap, always-on, agentic language core that can run per-team or per-engineer; our work supplies the efficient backbone such systems require.

III. ARCHITECTURE AND PIPELINE OVERVIEW

BaiZe is an end-to-end multimodal agentic system assembled in five stages: (i) from-scratch pre-training of a hybrid language backbone, (ii) post-training of that backbone (instruction tuning and reinforcement learning to strengthen agentic capability), (iii) pre-training of a custom vision encoder, (iv) alignment of the two into a multimodal large language model, and (v) semiconductor-domain adaptation via reinforcement learning. This section fixes the shared scaffolding—the two backbones, the overall pipeline, and the global training environment—before each stage is detailed in its own section.

A. Language Backbone

The language backbone is a Nemotron-H-style sequential hybrid model [8], [9]: 56 transformer layers interleave Mamba-2 state-space layers, four grouped-query-attention (GQA) layers, and MLP layers. Each Mamba-2 layer uses 8 state-space groups; the four attention layers use 16 query heads and 4 query groups (GQA) with untied word embeddings. The full configuration is listed in Table I; the model totals ≈ 2.22 B parameters.

TABLE I
BAI ZE LANGUAGE-BACKBONE CONFIGURATION (MAMBA2-HYBRID).

Hyper-parameter	Value
Transformer layers	56
Architecture	hybrid (Mamba-2 + GQA + MLP)
Attention layers	4 (interleaved)
Hidden size	2048
FFN hidden size	8192
Attention heads (Q)	16
Query groups (GQA)	4
Mamba-2 state-space groups	8
Vocabulary size	129,281
Sequence length	4096
Activation	SiLU / SwiGLU
Normalisation	RMSNorm
Precision	BF16
Parameters	≈ 2.22 B

B. Vision Encoder

The vision encoder is trained from scratch in stage (iii); its architecture and configuration are reported in Section VI ([TBD]).

C. Pipeline and Global Environment

Table II summarises the five-stage pipeline. All stages run on 8×NVIDIA H100 GPUs (CUDA 12.8, PyTorch 2.8.0,

Python 3.10); fused kernels, selective activation recomputation, and cross-entropy loss fusion are enabled where applicable, with `tensor_model_parallel_size = 1` and BF16 precision throughout.

TABLE II
BAIZE FIVE-STAGE PIPELINE OVERVIEW.

Stage	Task	Detailed in
(i)	LLM pre-training	Section IV
(ii)	LLM post-training (SFT + agentic RL)	Section V
(iii)	Vision encoder pre-training	Section VI
(iv)	MLLM alignment	Section VII
(v)	Semiconductor domain adaptation (RL)	Section VIII

IV. LLM PRETRAINING

Stage (i) pre-trains the hybrid language backbone (Section III-A) from scratch under a fixed budget. This section covers the architecture-selection rationale, the training recipe, and the resulting measurements; the backbone configuration itself is given in Table I.

A. Architecture Selection Rationale

We adopt the hybrid backbone on three grounds. **(i) Inference efficiency.** State-space layers decode with constant-time recurrence per token, while the sparse attention layers retain long-range context; this directly targets the inference-heavy, cost-sensitive deployments that motivate BaiZe. **(ii) Controlled comparison.** A matched from-scratch comparison at the $\approx 2\text{B}$ scale between a dense transformer (MiniCPM5-2B) and Mamba2-hybrid—trained under identical data, seed, and GPU footprint—favours the hybrid, which converges to a lower loss with fewer parameters ($\approx 2.22\text{B}$ vs. 2.51B , -11.6%) and decodes substantially faster, at a $\sim 24\%$ training-throughput cost (Table III). **(iii) Feasibility.** Megatron-Core ships a Mamba-hybrid provider whose 56-layer interleaving pattern is halved in width to reach the $\approx 2\text{B}$ scale, making the backbone directly executable under the NeMo recipe without custom kernels.

B. Training Framework and Reduced-Horizon Search

We train with the NVIDIA NeMo pre-training entry point, driving the Megatron-Core `NVIDIAMambaHybridModelProvider2B` provider directly with `tensor_model_parallel_size = 1`. To keep the entire sweep within budget, we run all hyper-parameter searches at a reduced iteration count and treat the resulting short-horizon ranking as predictive of the full-model optimum—a small-scale proxy strategy in the spirit of maximal-update parameterization [1]. Fused kernels, selective activation recomputation, and cross-entropy loss fusion are enabled to raise throughput.

C. Learning-Rate Schedule

BaiZe trains under a warmup–stable–decay-style learning-rate schedule whose hyper-parameters—warmup length, stable learning rate, and the decay ratio—are themselves swept in

Section IV-D (cosine decay is kept as an alternative). The remaining schedule and batch-size settings (global batch size, sequence length, packing) are held fixed across configurations and re-measured on the final backbone, with figures reported where applicable.

D. Budget-Constrained Search Protocol

Because a full grid is infeasible under a fixed GPU budget, we adopt a staged, single-variable-at-a-time protocol:

- **S1 (baseline).** Reproduce the Mamba2-hybrid reference configuration to establish a loss/throughput baseline.
- **S2 (hyper-parameter sweep).** Perturb one training variable at a time—stable learning rate, warmup length, and decay ratio—at a short horizon.
- **S3 (schedule sweep).** Compare learning-rate schedules (WSD-style vs. cosine) and batch sizing at a short horizon.
- **S4 (verification).** Retrain the winning configuration from scratch over a longer horizon to confirm convergence and reproducibility.

At every stage only a single variable changes relative to the running best, isolating each factor’s contribution while keeping total GPU-hours within budget. The architectural search space is validated up front: the hybrid (Mamba2-hybrid) and dense (MiniCPM5-2B) candidates are compared head-to-head from scratch (Section IV-A), which selects the hybrid backbone before the hyper-parameter search begins.

E. Data

Pre-training uses the Ultra-FineWeb-L3 English subset ($\approx 1.8\text{T}$ tokens), streamed and packed to a sequence length of 4096, tokenised with the DeepSeek-V4.1-Flash tokenizer (vocabulary size 129,281). Complementary corpora—code (UltraData-Code, 1.2T), mathematics (UltraData-Math, 515G), and an agentic instruction-tuning set (UltraData-SFT-Agent, 51G)—are reserved for post-training (Section V).

F. Search Budget

The pre-training search is bound by a ≈ 84 GPU-hour budget (aggregated across all experiments and retries). Short-horizon sweeps (S2, S3) run 80 steps (≈ 0.9 GPU-hours each); the baseline (S1) runs 300 steps and the verification run (S4) runs 160 steps. Validation loss is reported at matched step counts so that comparisons across configurations remain apples-to-apples.

G. Results

Following the architecture change to the hybrid backbone, all training and inference measurements are being re-executed on the final Mamba2-hybrid model; quantitative figures are reported as [TBD] and will be back-filled once re-measurement completes.

Table III contrasts the hybrid backbone against a dense transformer at the $\approx 2\text{B}$ scale, trained from scratch under matched data, seed, and GPU footprint; the hybrid offers

fewer parameters and faster decoding at a modest training-throughput cost.

TABLE III
ARCHITECTURE-LEVEL COMPARISON MOTIVATING THE HYBRID CHOICE ([TBD]: RE-MEASURED ON THE FINAL BACKBONE).

Metric	Dense (MiniCPM5-2B)	Hybrid (BaiZe)	Edge
Parameters	2.51B	2.22B	hybrid (−11.8%)
Final loss	[TBD]	[TBD]	[TBD]
Training tok/s	[TBD]	[TBD]	[TBD]
Prefill tok/s	[TBD]	[TBD]	[TBD]
Decode tok/s	[TBD]	[TBD]	[TBD]

Table IV reports the hyper-parameter search outcomes at a matched horizon; the searched axes target the hybrid training recipe (stable learning rate, warmup length, decay ratio, and schedule family).

TABLE IV
SEARCH OUTCOMES ([TBD]: RE-MEASURED ON THE FINAL HYBRID BACKBONE).

ID	Change vs. best	Val. loss	Outcome
S1-01	baseline (hybrid)	[TBD]	baseline
S2-01	learning-rate variant	[TBD]	[TBD]
S2-02	warmup variant	[TBD]	[TBD]
S3-01	schedule family	[TBD]	[TBD]
S4-01	winner, long horizon	[TBD]	winner

Training-loss curves and the short-horizon ablation visualisation are regenerated on the hybrid backbone and will be included as figures once re-measurement completes ([TBD]).

V. LLM POST-TRAINING

Stage (ii) strengthens the agentic capability of the pre-trained backbone through supervised instruction fine-tuning and reinforcement learning ([TBD]). This section will describe the fine-tuning corpus, the agentic reward design, and the post-training budget and results.

VI. VISION ENCODER PRETRAINING

Stage (iii) pre-trains a custom vision encoder from scratch ([TBD]). This section will detail the encoder architecture, pre-training data, and its configuration and results.

VII. MLLM ALIGNMENT

Stage (iv) aligns the vision encoder with the language backbone into a multimodal model, following a LLaVA-OneVision-style curriculum: stage-1 projector pre-training, stage-1.5 mid-training, and stage-2 instruction tuning ([TBD]). This section will detail each sub-stage’s data, objectives, and results.

VIII. SEMICONDUCTOR DOMAIN ADAPTATION

Stage (v) adapts the multimodal model to the semiconductor domain via reinforcement learning, targeting EDA tasks such as register-transfer-level understanding, verification, and bug triage ([TBD]). This section will describe the domain reward design, training setup, and EDA evaluation results.

IX. CONCLUSION

We presented BaiZe, an end-to-end multimodal agentic system: a ≈ 2.2 B-parameter hybrid (state-space + attention) language model built from scratch, post-trained for agentic capability, paired with a custom vision encoder, aligned into a multimodal model, and adapted to the semiconductor domain.

Through a controlled from-scratch comparison, we selected a Nemotron-H-style Mamba2-hybrid backbone over a dense transformer for its lower loss-per-parameter and substantially faster inference, then applied a staged, budget-constrained hyper-parameter search (S1–S4) on the final backbone. Training and inference measurements are being re-executed on the final model and are reported as [TBD]; the complete recipe and a single reproducible command encapsulate the entire construction under the NVIDIA NeMo recipe.

The immediate next step is agentic fine-tuning with the UltraData-SFT-Agent corpus toward a deployable agent core for EDA and embedded applications. We hope this blueprint lowers the barrier for cost-bounded teams building their own efficient agentic SLMs from scratch.

REFERENCES

- [1] G. Yang, E. J. Hu, I. Babuschkin, S. Sidor, X. Liu, D. Farhi, N. Ryder, J. Pachocki, W. Chen, and J. Gao, “Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer,” *arXiv preprint arXiv:2203.03466*, 2022.
- [2] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, Z. Zheng, Y. Fang, Y. Huang, W. Zhao *et al.*, “Minicpm: Unveiling the potential of small language models with scalable training strategies,” *arXiv preprint arXiv:2404.06395*, 2024.
- [3] A. Yang, B. Li, B. Zhang *et al.*, “Qwen3 technical report,” 2025.
- [4] G. Team, “Gemma 3 technical report,” 2025.
- [5] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang *et al.*, “Agentbench: Evaluating llms as agents,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [6] G. Yang, D. Yu, C. Zhu, and S. Hayou, “Tensor programs vi: Feature learning in infinite-depth neural networks,” *arXiv preprint arXiv:2310.02244*, 2023.
- [7] A. Gu and T. Dao, “Mamba: Linear-time sequence modeling with selective state spaces,” *arXiv preprint arXiv:2312.00752*, 2023.
- [8] T. Dao and A. Gu, “Transformers are ssms: Generalized models and efficient algorithms through structured state space duality,” *arXiv preprint arXiv:2405.21060*, 2024.
- [9] NVIDIA, “Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models,” 2025.
- [10] X. Dong, Y. Fu, S. Diao *et al.*, “Hymba: A hybrid-head architecture for small language models,” 2024.
- [11] K. Jordan, Y. Jin, V. Boza, J. You *et al.*, “Muon: An optimizer for hidden layers in neural networks,” 2025.
- [12] M. Shoenybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, “Megatron-lm: Training multi-billion parameter language models using model parallelism,” 2020.
- [13] M. Liu, T. E. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu *et al.*, “Chipnemo: Domain-adapted llms for chip design,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [14] J. Blocklove, R. Chapman, B. Tan, D. Chen *et al.*, “Chipbench: The llm chip design benchmark,” 2024.